

ML FRAMEWORKS

Adrian Jackson

a.jackson@epcc.ed.ac.uk



THE UNIVERSITY
of EDINBURGH



Frameworks

- Software packages that provide building blocks to construct and run DNN
 - DNNs have standard components they can be built from
 - Standard structures
 - Don't need developers to re-write these all the time
 - Particularly the backward propagation functionality (auto differentiation etc...)
- Provide higher level functionality to speed up the development, training, and deployment of DNNs
 - Like numerical libraries within computational simulation
 - Self managing functionality
- However, restricted subset of programming
 - Not as flexible as full programming language

Example frameworks

- PyTorch



- TensorFlow



- JAX



- Keras



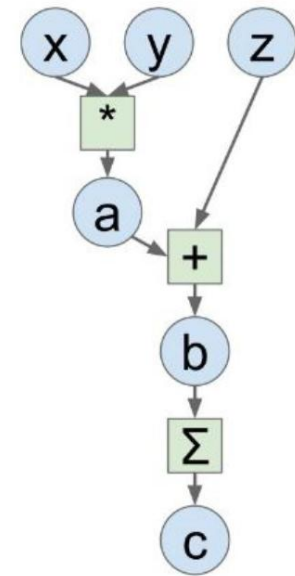
- Theano

theano

- Functionality to define, optimize, and evaluate mathematical expressions involving multi-dimensional arrays efficiently

PyTorch

- What is PyTorch?
 - An open-source machine learning library.
 - Developed by Facebook's AI Research lab (FAIR).
 - Widely used for deep learning research and applications.
- Main functionality
 - Dynamic computation graph
 - Pythonlike syntax, NumPy focussed
 - GPU Acceleration
 - Library functionality: TorchVision for image processing, TorchText for natural language processing, and TorchAudio for audio processing



PyTorch Tensors

- Tensors core data structure in PyTorch
 - N-dimensional matrices
 - Like NumPy ndarrays
 - Used for scalars up to nd-arrays, 1d is vector, 2d is matrix
 - Default to 32-bit floating point number (`torch.Tensor == torch.FloatTensor`)
- Initialise similar to NumPy:
 - `x = torch.empty(3, 4)`
 - `x = torch.zeros(3, 4)`
 - `x = torch.ones(3, 4)`
 - `torch.manual_seed(1729)`
 - `x = torch.rand(3, 4)`
- Can create with same shape as existing variable
 - `y = torch.zeros_like(x)`
- Can specify the datatype
 - `b = torch.rand((2, 3), dtype=torch.float64) * 20`
- PyTorch supports (<https://pytorch.org/docs/stable/tensors.html>):
 - `torch.bool`
 - `torch.int8`
 - `torch.uint8`
 - `torch.int16`
 - `torch.int32`
 - `torch.int64`
 - `torch.half`
 - `torch.bfloat16`
 - `torch.float`
 - `torch.double`

```
1 import torch
2
3 N, D = 3, 4
4
5 x = torch.rand((N, D), requires_grad=True)
6 y = torch.rand((N, D), requires_grad=True)
7 z = torch.rand((N, D), requires_grad=True)
8
9 a = x * y
10 b = a + z
11 c = torch.sum(b)
12
13 c.backward()
```

PyTorch Tensors

- Basic Operations

- Array operations

```
ones = torch.zeros(2, 2) + 1
twos = torch.ones(2, 2) * 2
threes = (torch.ones(2, 2) * 7 - 1) / 2
fours = twos ** 2
sqrt2s = twos ** 0.5
```

- Requires tensors to be the same size and shape
 - Exception is broadcasting operations

- Broadcasting

```
a = torch.ones(4, 3, 2)
b = a * torch.rand(3, 2)
```

- Each tensor must have at least one dimension - no empty tensors.
 - Comparing the dimension sizes of the two tensors, *going from last to first*:
 - Each dimension must be equal, or
 - One of the dimensions must be of size 1, or
 - The dimension does not exist in one of the tensors

```
tensor([[1., 1.],
        [1., 1.]])
tensor([[2., 2.],
        [2., 2.]])
tensor([[3., 3.],
        [3., 3.]])
tensor([[4., 4.],
        [4., 4.]])
tensor([[1.4142, 1.4142],
        [1.4142, 1.4142]])
```

PyTorch Tensors

- In-built functions

- Wide range of existing operations, i.e.

```
torch.abs(a)
```

```
torch.ceil(a)
```

```
torch.floor(a)
```

```
torch.clamp(a, -0.5, 0.5)
```

- In-place operations also possible

```
my_b.add_(my_a)
```

```
torch.sin_(b)
```

```
a_weights.mul_(gradients)
```

```
d = torch.matmul(a, b, out=c)
```

- Usual python/NumPy copying syntax

- `a = b` is just a reference

- `a = b.clone()` proper copy

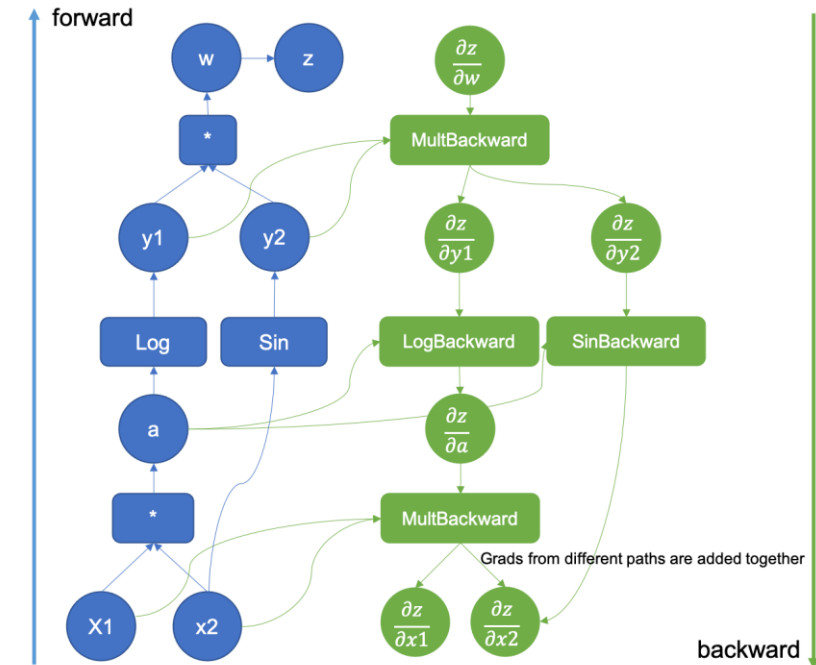
- `c = b.detach().clone` disables autograd tracking

Tensor gradients

- `requires_grad` - making a trainable parameter
 - Default: `False`
 - Turn on:
 - `my_tensor.requires_grad_()`
 - `my_tensor = torch.randn(1, requires_grad=True)`
 - Tracks all operations on the tensor, and gradients are computed during the backward pass
- Tensor value:
 - `my_tensor.data`
- Tensor gradient
 - `my_tensor.grad`
- Get gradient metadata
 - `my_tensor.grad_fn`

Autograd

- PyTorch's automatic differentiation engine
 - Backpropagation functionality
 - Gradients accumulated at each step
- Computational graph built automatically on the forward pass
 - Nodes are tensors or operations
 - edges are outputs of nodes
- Forward data automatically freed once gradients have been computed
`y.backward(torch.ones(2, 2), retain_graph=True)`
- Need to zero out gradients after each step
 - Gradients are accumulated across epochs by default
`optimizer.zero_grad()`
- Custom Gradients
 - Can specify custom gradient calculations in the `backwards()` function
 - Not restricted to the default auto calculations
- Requires memory to store the data for gradient operations during the forward pass



```
# Create tensors.
x = torch.tensor(1., requires_grad=True)
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(3., requires_grad=True)

# Build a computational graph.
y = w * x + b    # y = 2 * x + 3

# Compute gradients.
y.backward()

# Print out the gradients.
print(x.grad)    # x.grad = 2
print(w.grad)    # w.grad = 1
print(b.grad)    # b.grad = 1
```

Tensors

- Change size/shape of tensors

- `squeeze` and `unsqueeze` change the number of dimensions
- Removing/adding dimension of extent 1
- `reshape` changes shape of tensor

- PyTorch tensors GPU capable

- `torch.cuda.is_available()`
- Data created on CPU by default

```
gpu_rand = torch.rand(2, 2, device='cuda')  
y = torch.rand(2, 2)  
y = y.to(my_device)
```

- Cannot mix operations on CPU and GPU data

- Can create a default device to use everywhere:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")  
model = NeuralNetwork().to(device)
```

```
a = torch.rand(3, 226, 226)  
b = a.unsqueeze(0)
```

```
print(a.shape)  
print(b.shape)
```

```
torch.Size([3, 226, 226])  
torch.Size([1, 3, 226, 226])
```

Neural network construction

- `torch.nn` building blocks for neural networks
 - Includes basic functionality such as activation functions and layers
- Two main parts defining a network:
 - `__init__(self)`: defines the parts that make up the model
 - `forward(self, x)`: defines the forward computation
- Layers
 - Linear (Fully Connected) Layers: `nn.Linear`
 - Convolutional Layers: `nn.Conv2d` for image processing
 - Recurrent Layers: `nn.LSTM` or `nn.GRU` for sequence data
- Activation Functions:
 - Wide range of activation functions and normalization layers
- Initialise weights
 - Need to specify the initial values of weights, biases, etc...
 - Uniform, random, etc...

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out
```

Optimiser

- Optimisation step to train network
- PyTorch has a range of in-built optimiser
 - `torch.optim` object
 - SGD, Adam, RMSProp, etc...
- Need to specify a loss function
 - `nn` has a range of loss functions
 - Mean Absolute Error Loss (L1), Mean Squared Error Loss (L2), Negative Log-Likelihood Loss, Cross-Entropy Loss, Hinge Embedding Loss, Margin Ranking Loss, Triplet Margin Loss, Kullback-Leibler divergence, etc...
 - Build loss function from output/target and then use in the optimiser
- Also can specify loss rate schedulers etc...
 - Dynamic loss rate across epochs

```
a = torch.randn(1, requires_grad=True, dtype=torch.float, device=device)
b = torch.randn(1, requires_grad=True, dtype=torch.float, device=device)

# Defines a SGD optimizer to update the parameters
optimizer = optim.SGD([a, b], lr=lr)

for epoch in range(n_epochs):
    yhat = a + b * x_train_tensor
    error = y_train_tensor - yhat
    loss = (error ** 2).mean()

    loss.backward()

    optimizer.step()

    optimizer.zero_grad()
```

Data loading

- `torch.utils.data.Dataset`
 - uniform interface for different data types.
 - Predefined classes for different functionality (i.e. `ImageFolder`)
- `torch.utils.data.DataLoader`
 - Provides iterable interface over data
 - Gives batching, shuffling, and parallel loading functionality

```
from torch.utils.data import DataLoader

from torch.utils.data import Dataset

from torchvision import datasets

from torchvision.transforms import ToTensor

import matplotlib.pyplot as plt

training_data = datasets.FashionMNIST(root="data", train=True, download=True, transform=ToTensor())

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
```

PyTorch 2.5

- New backend for scaled dot product attention (SDPA)
 - Important layer for LLMs, i.e. `torch.nn.MultiheadAttention`, `torch.nn.TransformerEncoderLayer`
 - Multiple available backend implements, new version `Fused Flash Attention`
- Improvements in Compiled autograd functionality
 - Fully backwards graph capture
- Flight recorder debugger
 - Records distributed training activities
 - Identify deadlocks

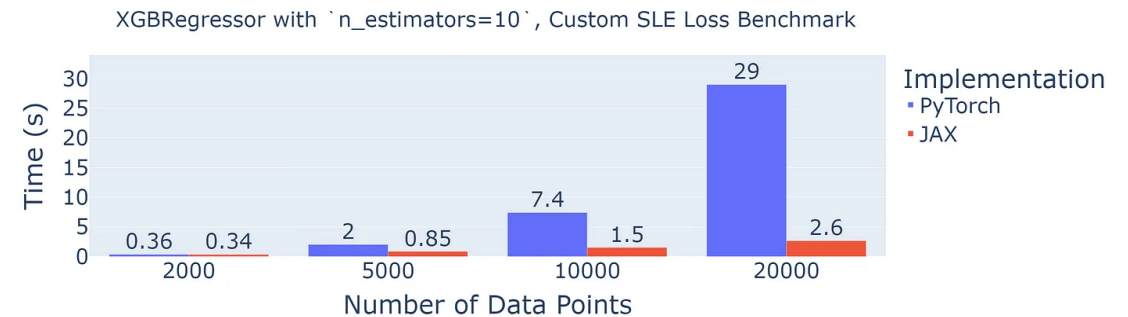
Lightning

- PyTorch Lightning is an abstraction/functionality layer built on PyTorch
 - Separate code/repository
 - Uses PyTorch for functionality
 - Simplifies model development and scaling
 - Separates training and model definition

```
model = AdvancedClassifier(learning_rate=0.001)
trainer = pl.Trainer(
    accelerator='gpu',
    devices=4,  # Use 4 GPUs
    strategy='ddp',  # Distributed Data Parallel
    precision=16  # Mixed precision
)
trainer.fit(model, train_dataloader)
```

JAX

- Numerical library, similar to NumPy functionality
- Uses XLA for computational graph etc...
- Can do autograd/differentiation
 - Powerful auto-differentiation functionality
 - High performance for certain autograd functionality (i.e. high order)
 - Although PyTorch and Tensorflow with XLA may give similar performance
- Functional style
 - A bit lower level than alternatives
 - More direct control than PyTorch



JAX

- Defaults to accelerator computation, falls back to CPU
- Combined kernel dispatch important JAX optimisation
 - JIT functionality to combine code

```
from jax import random
def selu(x, alpha=1.67, lambda=1.05):
    return lambda * jnp.where(x > 0, x, alpha * jnp.exp(x) - alpha)
key = random.key(1701)
x = random.normal(key, (1_000_000,))
selu(x).block_until_ready()
```

```
from jax import jit
selu_jit = jit(selu)
selu_jit(x).block_until_ready()
```

- JAX uses asynchronous dispatch by default
 - Returns Python `Future` for operations

JAX

- Also provides automatic vectorisation (`vmap`) alongside the compilation stuff (`jit`)

```
import jax
import jax.numpy as jnp
x = jnp.arange(5)
w = jnp.array([2., 3., 4.])
def convolve(x, w):
    output = []
    for i in range(1, len(x)-1):
        output.append(jnp.dot(x[i-1:i+2], w))
    return jnp.array(output)
convolve(x, w)

xs = jnp.stack([x, x])
ws = jnp.stack([w, w])
auto_batch_convolve = jax.vmap(convolve)
auto_batch_convolve(xs, ws)
```

JAX

- Automatic differentiation supported using the `grad` function

```
import jax
import jax.numpy as jnp
from jax import grad
grad_tanh = grad(jnp.tanh)
```

- Can put all these together to make custom DNNs

```
def one_hot(x, k, dtype=jnp.float32):
    """Create a one-hot encoding of x of size k."""
    return jnp.array(x[:, None] == jnp.arange(k), dtype)

def accuracy(params, images, targets):
    target_class = jnp.argmax(targets, axis=1)
    predicted_class = jnp.argmax(batched_predict(params, images), axis=1)
    return jnp.mean(predicted_class == target_class)

def loss(params, images, targets):
    preds = batched_predict(params, images)
    return -jnp.mean(preds * targets)

@jit
def update(params, x, y):
    grads = grad(loss)(params, x, y)
    return [(w - step_size * dw, b - step_size * db)
            for (w, b), (dw, db) in zip(params, grads)]
```

Summary

- Large scale frameworks for ML generally follow the same approaches
 - Underlying graph computation
 - Based on Tensors
 - Functionality for GPUs
 - AutoGrad operations
- Some structural differences between them
 - The level of detail exposed
 - The control/functionality
 - Type of defined functionality
- Useful to be able to use at least 2 different frameworks well to fully understand ML DNNs and how they work with computing hardware